Aula 03 - Protegendo o Sistema - Encapsulamento (Getters e Setters)



Projeto: FiapRide (Sistema de Mobilidade Urbana) + Seu Projeto Pessoal

Foco: Java, Segurança de Dados, Clean Code

🎯 Objetivo da Aula

Na aula passada, nós criamos o método `adicionarSaldo()`. Porém, se você for no seu `main` agora e digitar `passageiro1.saldo = -500.0;`, o Java ainda vai permitir! Isso acontece porque nossos dados estão "nus" e desprotegidos.

Hoje, vamos aprender a técnica de **Encapsulamento**. Você vai fechar as portas do seu objeto e controlar rigorosamente quem pode ler ou alterar as informações dele.

Conceitos Chave:

- **Encapsulamento:** Esconder os detalhes internos de como um objeto funciona e proteger seus dados.
- **Modificadores de Acesso:** Palavras-chave que definem quem pode ver o quê (`public` e `private`).
- **Getters e Setters:** Métodos padronizados para pegar (`get`) e alterar (`set`) valores de forma segura.

📖 A Teoria: A Cápsula e o Caixa Eletrônico

Pense em um Caixa Eletrônico (ATM). O cofre cheio de dinheiro dentro dele é **privado** (*private*). Você não pode simplesmente abrir a tampa e pegar as notas. O banco te fornece uma tela com botões (que são **públicos** - *public*).

Você clica em "Sacar", o sistema verifica seu saldo (Regra de Negócio) e, se estiver tudo certo, a máquina mexe no cofre por você.

Na programação é igual: **Nunca deixe outras classes mexerem diretamente nos atributos do seu objeto!**

🛠️ Parte 1: Modelagem Visual (Astah)

Vamos atualizar nossa planta para refletir essa segurança.

1. Abra o **Astah** e selecione a classe `Passageiro`.
2. Dê um duplo clique nos atributos `nome`, `saldo` e `cpf`.
3. Mude a visibilidade deles:

- Agora, mude para um símbolo de menos `-` (Private).

1. Adicione os métodos de acesso (Operations) com o símbolo `+` (Public) e `-` (Private):

- `+ getSaldo(): double`
- `- setSaldo(valor: double) : void`
- `+ getNome(): String`
- `- setNome(nome: String) : void`
- `+ getCpf(): String`
- `- setCpf(cpf: String) : void`



💻 Parte 2: O Desafio FiapRide (Mão na Massa)

Vamos aplicar o encapsulamento na nossa classe `Passageiro`.

Passo 1: Trancando as Portas (Modificador Private)

Vá na classe `Passageiro` e troque a palavra `public` dos atributos por `private`.

```
package br.com.fiapride.model;

public class Passageiro {

    // Agora ninguém fora desta classe consegue mexer nisso diretamente!
    private String nome;
    private String cpf;
    private double saldo;

    // ... métodos anteriores (adicionarSaldo) continuam aqui
}
```

⚠️ ***Atenção:*** *Se você olhar o seu arquivo `SistemaPrincipal.java` agora, ele estará cheio de erros vermelhos. Isso é ótimo! Significa que o encapsulamento funcionou e barrou o acesso direto.*

Passo 2: Criando as Portas de Acesso (Get e Set)

Ainda na classe `Passageiro`, vamos criar os métodos para acessar esses dados com segurança.

```
    // CONSTRUTOR (Obrigatório no Nascimento)
    // Para criar um passageiro, agora é OBRIGATÓRIO informar nome e cpf.
    public Passageiro(String nome, String cpf) {
        this.setNome(nome);
        this.setCpf(cpf);
        this.setSaldo(0); // Todo mundo começa com zero!
    }

    // Método para PEGAR o saldo (Leitura)
    public double getSaldo() {
        return this.saldo; // Apenas devolve o valor, não altera nada.
    }

    // Método para DEFINIR o saldo (Escrita com Regra de Negócio!)
    private void setSaldo(double valor) {
        if (valor >= 0) {
            this.saldo = valor;
        } else {
            System.out.println("Erro de Segurança: Tentativa de definir saldo negativo bloqueada!");
        }
    }

    // Faça o mesmo para o nome (sem regras complexas por enquanto)
    public String getNome() {
        return this.nome;
```

```
    }

    private void setNome(String nome) {
        this.nome = nome;
    }

    // Faça o mesmo para o cpf (sem regras complexas por enquanto)
    public String getCpf() {
        return this.cpf;
    }

    private void setCpf(String cpf) {
        this.cpf = cpf;
    }
```

🧠 **Entendendo o Código (Clean Code):**

Notou que os nossos `Setters` (`setNome` , `setSaldo`) são **PRIVATE**? Isso é uma prática avançada! Em vez de deixar o programa alterar o saldo livremente usando um `setSaldo(50)` , nós obrigamos o programa a usar métodos com regras de negócio claras: `adicionarSaldo()` e `pagarViagem()` .

Passo 3: Consertando o Main (O Teste)

Agora vamos ver como o `main` interage com nosso objeto blindado. O `main` não pode mais tocar nos atributos, ele precisa "pedir" para a classe Passageiro fazer o trabalho.

Abra sua classe `SistemaPrincipal` e atualize:

```
package br.com.fiapride.main;

import br.com.fiapride.model.Passageiro;

public class SistemaPrincipal {

    public static void main(String[] args) {

        System.out.println("--- Iniciando o Sistema FiapRide --- \n");

        // INSTANCIAÇÃO (Usando o Construtor!)
        // Não precisamos mais de passageiro.nome = "Ana", já passamos tudo de uma vez.
        Passageiro passageiro1 = new Passageiro("Ana Silva", "222.222.222-22");
        System.out.println(">>> Recarga Passageiro 1:");
        passageiro1.adicionarSaldo(50.0);

        Passageiro passageiro2 = new Passageiro("Carlos Souza", "333.333.333-33");
        System.out.println(">>> Recarga Passageiro 2:");
        passageiro2.adicionarSaldo(12.5);

        System.out.println("\n--- Status dos Passageiros ---");
        // Lendo os dados com os Getters:
```

```
        System.out.println("Passageiro: " + passageiro1.getNome() + " | Saldo: R$ " + passageiro1.ge
tSaldo() + " | CPF: " + passageiro1.getCpf());
        System.out.println("Passageiro: " + passageiro2.getNome() + " | Saldo: R$ " + passageiro2.ge
tSaldo() + " | CPF: " + passageiro2.getCpf());

        System.out.println("\n--- Realizando Viagens ---");
        System.out.println("Pagando viagem do passageiro 1 (Ana)...");
        passageiro1.pagarViagem(20.0); // Ana tem 50, vai sobrar 30.

        System.out.println("\nPagando viagem do passageiro 2 (Carlos)...");
        passageiro2.pagarViagem(20.0); // Carlos tem 12.5. O sistema DEVE bloquear!

        // Tente hackear o sistema descomentando a linha abaixo:
        // passageiro1.saldo = 999999.0; // O Java vai exibir erro vermelho! A proteção funcionou!
    }
}
```

🧠 Pergunta de Reflexão

No nosso código, os atributos são `private`, mas os métodos `getSaldo()` e `getNome()` são `public`. Por que é seguro deixar o `get` público, mas perigoso deixar o atributo original público?

Pense bem: Qual a diferença entre dar a alguém uma CÓPIA de um documento seu, e entregar o documento ORIGINAL para a pessoa rasurar?

🗂️ Organizando a Casa: Separando seu Projeto Pessoal

Até agora, testamos nosso objeto pessoal e o FiapRide no mesmo arquivo `SistemaPrincipal`. Conforme o código cresce, o console vira uma bagunça!

Na Engenharia de Software, usamos a **Separação de Responsabilidades**. Vamos criar um arquivo executável (`main`) APENAS para o seu projeto pessoal, deixando o `SistemaPrincipal` exclusivo para o FiapRide.

Passo a Passo: Criando o Executável do seu Objeto

1. No "Package Explorer" (lado esquerdo do Eclipse), abra a pasta do seu projeto.
2. Clique com o botão direito no pacote `br.com.fiapride.main`.
3. Vá em **New > Class**.
4. **Name:** Digite `TesteMeuObjeto` (ou `Teste` + o nome do seu objeto, ex: `TesteGarrafa`).
5. **Importante:** Marque a caixinha `public static void main(String[] args)`.
6. Clique em **Finish**.

Movendo o Código:

Agora, você tem um espaço limpo só para você. Siga estes passos:

1. Vá no arquivo `SistemaPrincipal.java`.
2. **Recorte** (Ctrl+X) todo o código de teste do seu objeto pessoal (instanciação).
3. Vá no novo arquivo `TesteMeuObjeto.java` e **Cole** (Ctrl+V) dentro do método `main`.

Seu novo arquivo deve ficar parecido com isso (exemplo usando uma Garrafa):

```
package br.com.fiapride.main;

// Lembre-se de importar a sua classe do pacote model!
import br.com.fiapride.model.Garrafa;

public class TesteMeuObjeto {

    public static void main(String[] args) {

        System.out.println("--- Teste do Meu Projeto Pessoal ---");

        // 1. Instanciando o objeto (criando o espaço na memória)
        Garrafa minhaGarrafa = new Garrafa();

        // 2. Definindo os atributos diretamente
        minhaGarrafa.cor = "Azul";
        minhaGarrafa.capacidade = 500;

        // 3. Testando algum comportamento (se você criou na Aula 02)
        // minhaGarrafa.encher();

        // 4. Imprimindo o resultado no console
        System.out.println("Cor da garrafa: " + minhaGarrafa.cor);
        System.out.println("Capacidade: " + minhaGarrafa.capacidade + "ml");

    }
}
```

🏃 Como rodar apenas o seu projeto?

No Eclipse, se você clicar no botão verde de "Play", ele pode rodar o último arquivo aberto (o FiapRide). Para garantir que você está rodando o teste do SEU objeto:

- Deixe a aba do `TesteMeuObjeto.java` aberta e selecionada.
- Clique no botão **Play** ou use o atalho `Ctrl + F11`.

⚡ Microdesafio Pessoal: Encapsule o seu Objeto!

Lembra do objeto pessoal que você programou nas aulas passadas (Garrafa, Tênis, Smartphone, Pet)?

Baixe o seu projeto do Git seguindo o tutorial da Aula 2, caso precise de apoio pode usar os passos a passos a seguir:

- Acesso SSH ao GIT.
- Como consertar seu repositório no GitHub.

Sua Missão:

1. Atualize o Astah com os novos métodos e atributos.
2. Não esqueça de salvar o .asta na raiz do projeto para versionar.

3. Vá na classe do seu objeto e transforme todos os atributos em `private`.
4. Crie os métodos `get` e `set` para eles.
5. **Regra Especial:** Escolha pelo menos UM atributo e crie uma regra no `set` dele.
   - *Exemplo (Garrafa):* O `setCapacidade` não pode aceitar valores menores que 0 ou maiores que 3000 ml.
   - *Exemplo (Smartphone):* O `setArmazenamento` não pode aceitar valores negativos.
6. Vá no seu `main`, teste a regra tentando burlar o sistema e prove que sua classe está protegida.

🧼 Checklist: Clean Code (Boas Práticas)

- [ ] **Atributos SEMPRE privados:** A regra de ouro. Todo atributo de classe começa como `private`.
- [ ] **Padrão de Nomenclatura:** Getters e Setters devem seguir a regra do CamelCase: `getNomeDoAtributo()` e `setNomeDoAtributo()`.
- [ ] **Não crie Setters cegamente:** Uncle Bob (Robert C. Martin) nos avisa: não saia criando `set` para tudo. Se um atributo nunca deve ser alterado depois de criado (como o tamanho de uma TV), ele NÃO DEVE ter um método `setTamanho()`. Exponha apenas o que for estritamente necessário.

📤 Etapa Final: Versionamento (GitHub)

Sempre salve seu progresso. No terminal do seu projeto:

```
git add .
git commit -m "refactor: aplica encapsulamento na classe Passageiro e Objeto Pessoal"
git push origin main
```

Bons estudos!


FIAP
GRADUAÇÃO
Copyright © 2026 Prof. Ygor Anjos
Todos direitos reservados. Reprodução ou divulgação total ou parcial deste documento é expressamente proibido sem o consentimento formal, por escrito, do Professor (autor).